

ReactJS

Routing in ReactJS:

Routing is the mechanism that allows a web application to change what the user sees on the screen based on the URL in the browser's address bar.

What is a Route?

A Route is a specific URL path mapped to a specific component or view in your application.

Think of it as a set of instructions for your app:

- If the user visits /, show the Home component.
- If the user visits /dashboard, show the Dashboard component.
- If the user visits /settings, show the Settings component.

Why Do We Need It?

By default, React is a Single Page Application (SPA) framework. This means the browser loads exactly *one* HTML file (index.html) from the server. **Without routing**, if a user wanted to see their profile, you would have to manage that using state (e.g., if (showProfile) render Profile()). However, this introduces **major issues**:

- **No Unique URLs:** The URL would always just be www.myapp.com. Users couldn't bookmark their profile or share a specific page link with a friend.
- **Broken Back/Forward Buttons:** Clicking the browser's back button would take the user completely out of your website, rather than just taking them back to the previous view.
- **Poor SEO:** Search engines struggle to index different sections of your app if they all live under a single, unchanging URL.

Routing solves all of this by syncing your app's UI with the browser's URL bar.

Why You Should Prefer react-router-dom?

react-router-dom is the undisputed industry standard for React routing for several key reasons:

- **Declarative Routing:** It allows you to define your routes using standard React components and hooks, fitting perfectly into the React ecosystem.
- **Dynamic Matching & Parameters:** It easily handles complex URLs like /user/:id (where :id can be any user's ID).
- **Nested Layouts:** It lets you keep a persistent sidebar or navbar while only swapping out the center content of your page.
- **Performance Optimization:** It includes built-in features for code-splitting and lazy-loading, ensuring your app stays fast as it grows.

Command to install the react-router-dom:

```
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> npm i react-router-dom

added 4 packages, and audited 140 packages in 4s

32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

To import:

```
import { BrowserRouter, RouterProvider } from 'react-router-dom';
```

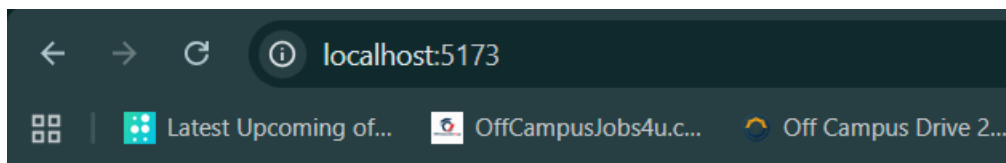
Now, let's understand this with an example:

App.jsx:

```
src > App.jsx > App
1 import React from 'react';
2 import { BrowserRouter, RouterProvider } from 'react-router-dom';
3 import Home from './myComponent/Home';
4 import About from './myComponent/About';
5 import Dashboard from './myComponent/Dashboard';
6
7 const router = createBrowserRouter([
8   { path: "/", element: <Home /> },
9   { path: "/about", element: <About /> },
10  { path: "/dashboard", element: <Dashboard /> },
11 ]);
12
13 function App() {
14   return (
15     <div>
16       <RouterProvider router={router} />
17     </div>
18   );
19 }
20
21 export default App;
```

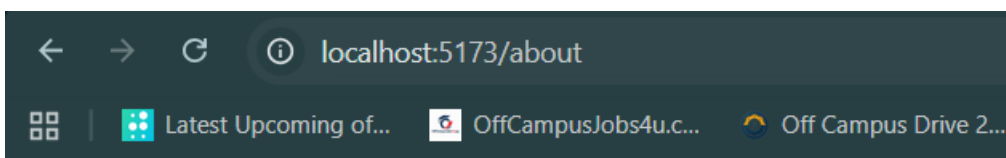
Home.jsx, About.jsx, Dashboard.jsx: has basic code inside it.

Output:



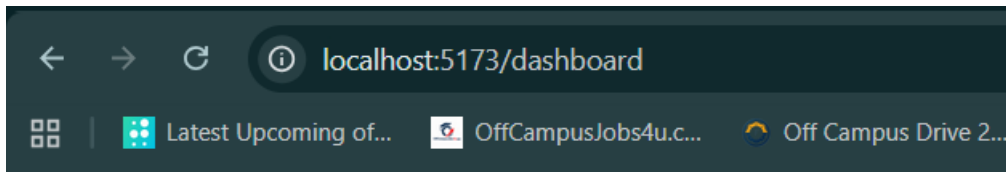
Home page

Output: when we hit the url /about



About page

Output: when we hit the url /dashboard



Dashboard page

Clearly, it worked as intended. Now, let's understand the part of that code:

```
const router = createBrowserRouter([
  { path: "/", element: <Home /> },
  { path: "/about", element: <About /> },
  { path: "/dashboard", element: <Dashboard /> },
]);
```

This function is used to declare all your application paths and their corresponding components in a clean, nested structure.

This code creates a routing configuration for your React app. `createBrowserRouter()` takes an array of route objects, where each object says: "If the URL matches this path, show this component." For example, if the URL is `/`, React renders `<Home />`; if it's `/about`, it renders `<About />`; and if it's `/dashboard`, it renders `<Dashboard />`. The resulting router object is then given to `<RouterProvider>` so React Router knows how to navigate between pages without reloading the browser.

While, this code part:

```
<div>
  <RouterProvider router={router} />
</div>
```

This component is wrapped around your root application. It actually kicks off the routing functionality and injects the route configurations into React.

`<RouterProvider router={router} />` is the component that activates the router in your app. It takes the router object you created with `createBrowserRouter()` and tells React, "Use these route rules to decide which component to display based on the current URL." Without `RouterProvider`, your routes exist as configuration, but React wouldn't know how to use them for navigation.

But, don't you think we have to manually enter the address in the url? Yes, to make sure we don't do this, we will use the `Navbar`. For which we created a `Navbar.jsx`:

```
src > myComponent > Navbar.jsx > Navbar
1  import React from 'react'
2  import { Link } from 'react-router-dom';
3  function Navbar() {
4    return (
5      <div>
6        <ul>
7          <li>
8            <Link to="/">Home</Link>
9          </li>
10         <li>
11           <Link to="/about">About</Link>
12         </li>
13         <li>
14           <Link to="/dashboard">Dashboard</Link>
15         </li>
16       </ul>
17     </div>
18   )
19 }
20 export default Navbar
```

Now, we have to make sure that Navbar shall be present in each page, so we have to change the App.jsx, making sure that for each url, Navbar shall be present.

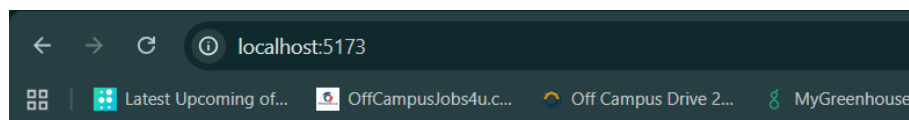
For, that the part will look like this:

```
const router = createBrowserRouter([
  { path: "/", element: <div><Home /><Navbar/></div> },
  { path: "/about", element: <div><About /><Navbar/></div> },
  { path: "/dashboard", element: <div><Dashboard /><Navbar/></div> },
]);
```

Now, the whole App.jsx:

```
src > App.jsx > App
1  import React from 'react';
2
3  import { createBrowserRouter, RouterProvider } from 'react-router-dom';
4
5  import Home from './myComponent/Home';
6  import About from './myComponent/About';
7  import Dashboard from './myComponent/Dashboard';
8  import Navbar from './myComponent/Navbar';
9
10 const router = createBrowserRouter([
11   { path: "/", element: <div><Home /><Navbar/></div> },
12   { path: "/about", element: <div><About /><Navbar/></div> },
13   { path: "/dashboard", element: <div><Dashboard /><Navbar/></div> },
14 ]);
15
16 function App() {
17   return (
18     <div>
19       <RouterProvider router={router} />
20     </div>
21   );
22 }
23
24
25 export default App;
```

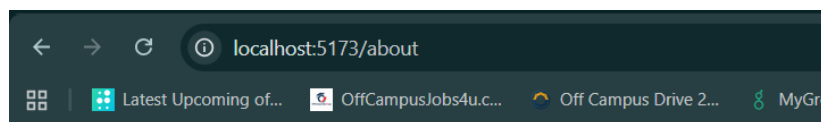
Output:



Home page

- [Home](#)
- [About](#)
- [Dashboard](#)

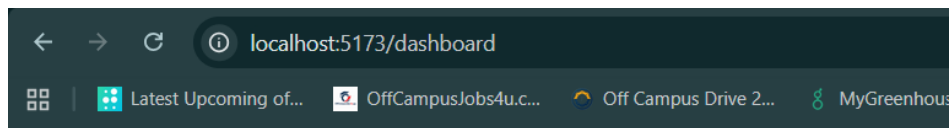
Output:



About page

- [Home](#)
- [About](#)
- [Dashboard](#)

Output:



Dashboard page

- [Home](#)
- [About](#)
- [Dashboard](#)

Observe that the page is not reloading. Fulfilling the goal of SPA of the react. But what actually happening in the background?

The browser does not reload the entire page. Instead:

1. React Router changes the URL using the browser's History API.
2. It checks which route matches the new URL.
3. It unmounts the old route component (for example, `<Home />`).
4. It mounts the new route component (for example, `<About />`).
5. Components that are outside the changing area (like a Navbar) remain on the screen and are not recreated.

Now, you would have observed some kind of parameters passed in the URL when we visit a link? Right? Yes!

What are the parameters?

In routing, parameters (often called URL params or dynamic routes) are placeholders in a URL path that allow you to capture dynamic values. Instead of creating a unique route for every single user, product, or article (e.g., `/user/1`, `/user/2`, `/user/3`), you define one single route with a parameter placeholder.

How to Define a Route Parameter?

In react-router-dom, you define a parameter by placing a colon (:) in front of the parameter name inside your path string.

Example:

```
{path: "/student/:id", element: <div><ParamComp/><Navbar/></div>}
```

Now, the App.jsx:

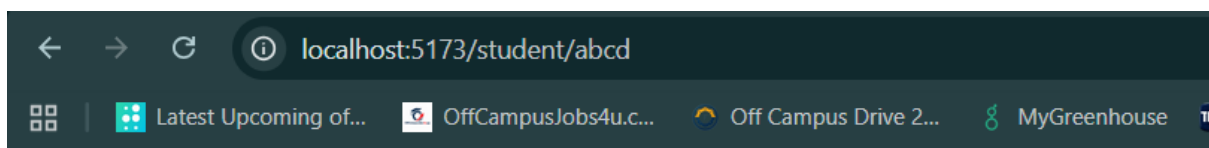
```
src > App.jsx > App
1 import React from 'react';
2
3 import { createBrowserRouter, RouterProvider } from 'react-router-dom';
4
5 import Home from './myComponent/Home';
6 import About from './myComponent/About';
7 import Dashboard from './myComponent/Dashboard';
8 import Navbar from './myComponent/Navbar';
9 import ParamComp from './myComponent/ParamComp';
10
11 const router = createBrowserRouter([
12   { path: "/", element: <div><Home /><Navbar/></div> },
13   { path: "/about", element: <div><About /><Navbar/></div> },
14   { path: "/dashboard", element: <div><Dashboard /><Navbar/></div> },
15   { path: "/student/:id", element: <div><ParamComp/><Navbar/></div> }
16 ]);
17
18 function App() {
19   return (
20     <div>
21       <RouterProvider router={router} />
22     </div>
23   );
24 }
25
26
27 export default App;
```

And the new component which we created, Paramcomp.jsx:

```
src > myComponent > ParamComp.jsx > ...
1 import React from 'react'
2 import { useParams } from 'react-router-dom'
3
4 function ParamComp() {
5   const {id} = useParams();
6   return (
7     <div>
8       Params: {id}
9     </div>
10   )
11 }
12
13 export default ParamComp
```

To access the actual value typed into the URL inside your component, React Router provides a built-in hook called useParams.

Output: the moment we type /student/abcd in the URL.



Params: abcd;

- [Home](#)
- [About](#)
- [Dashboard](#)

Now, there is something called as useNavigate hook.

What is useNavigate hook?

The useNavigate hook is used for programmatic navigation. It allows you to redirect users to different pages using code logic (e.g., inside functions, event handlers, or useEffect blocks) rather than relying on a direct user click on a <Link> component.

When to Use useNavigate vs. <Link>

- Use <Link> (Declarative): For static navigation where the user directly initiates the change by clicking (e.g., Navbars, Menus, Footers).
- Use useNavigate (Programmatic): For dynamic navigation triggered by background code or after an event finishes executing.

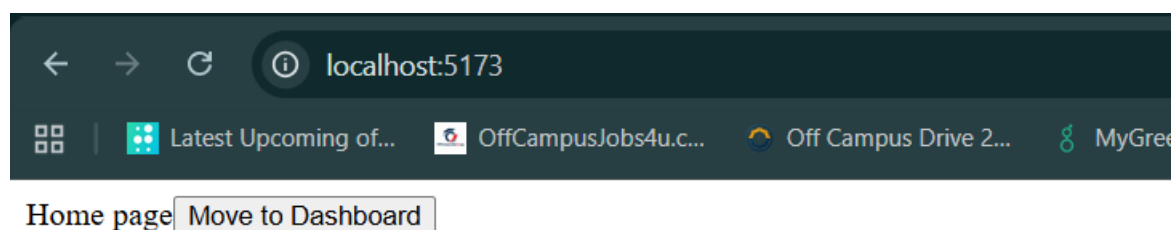
Example: suppose we add a button on the home page to navigate to the dashboard.

Home.jsx:

```
src > myComponent > Home.jsx > default
1  import React from 'react'
2  import { useNavigate } from 'react-router-dom'
3
4  function Home() {
5    const navigate = useNavigate();
6    function handleClick(){
7      navigate('/dashboard')
8    }
9    return (
10     <div>
11       Home page
12       <button onClick={handleClick}>Move to Dashboard</button>
13     </div>
14   )
15 }
16
17 export default Home
```

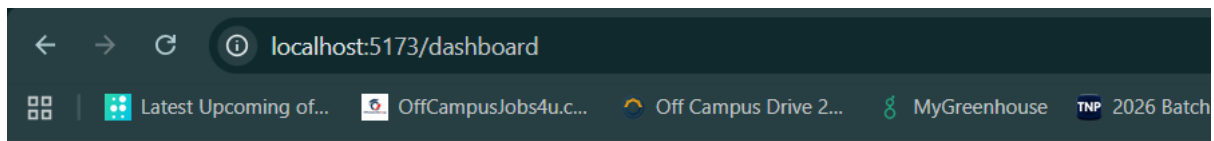
useNavigate() provides a navigate function that lets you change routes using JavaScript. In this code, when handleClick() is called (for example, by clicking a button), navigate('/dashboard') changes the URL to /dashboard and displays the Dashboard component without reloading the entire page.

Output:



- [Home](#)
- [About](#)
- [Dashboard](#)

Output: when clicked on the button.



Dashboard page

- [Home](#)
- [About](#)
- [Dashboard](#)

Done!

Now, we will study about the concept of Nested routing:

What is Nested Routing?

Nested Routing is the practice of rendering routes *inside* other routes. It allows you to build complex user interfaces where parts of the page stay exactly the same (like a sidebar, layout header, or dashboard menu) while other parts of the page dynamically swap content based on the sub-path.

Think of an application dashboard. The top navigation bar and left sidebar should stay on the screen at all times. When a user switches between "Analytics" and "Settings," only the center content panel changes.

- URL /dashboard -> Shows Dashboard Layout + General Stats
- URL /dashboard/analytics -> Shows Dashboard Layout + Live Graphs
- URL /dashboard/settings -> Shows Dashboard Layout + Settings Forms

How to set up Nested routing?

To set up nested routing using `createBrowserRouter`, you pass an array of routes to the `children` property of a parent route.

Example:

App.jsx:

```
src > App.jsx > default
1  import React from 'react';
2  import { createBrowserRouter, RouterProvider } from 'react-router-dom';
3  import Home from './myComponent/Home';
4  import About from './myComponent/About';
5  import Dashboard from './myComponent/Dashboard';
6  import Navbar from './myComponent/Navbar';
7  import ParamComp from './myComponent/ParamComp';
8
9  const router = createBrowserRouter([
10   { path: "/", element: <div><Navbar /><Home /></div> },
11   { path: "/about", element: <div><Navbar /><About /></div> },
12   { path: "/dashboard", element: <div><Navbar /><Dashboard /></div>, children: [{ index: true, element: <div>Overview Content</div> },
13     { path: "settings", element: <div>Settings Content</div> } ] },
14   { path: "/student/:id", element: <div><Navbar /><ParamComp /></div> }
15 ]);
16
17 function App() {
18   return <div><RouterProvider router={router} /></div>;
19 }
20
21 export default App;
```

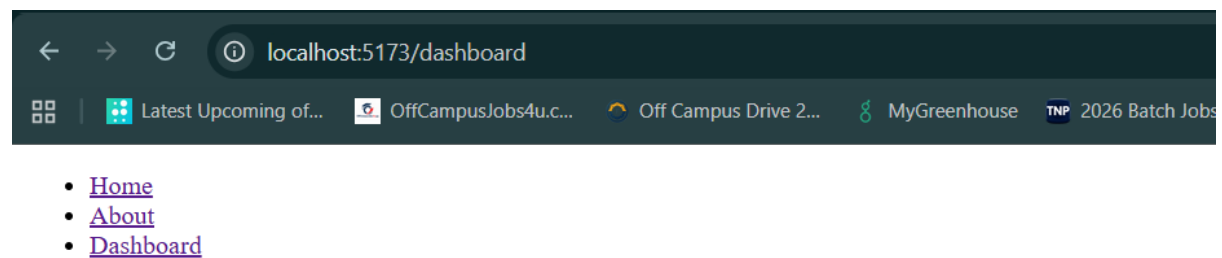
It defines /dashboard as a parent layout containing a navigation bar and the main dashboard view, which automatically loads "Overview Content" at the main URL or "Settings Content" when navigating to /dashboard/settings.

Dashboard.jsx:

```
src > myComponent > Dashboard.jsx > default
1  import React from 'react';
2  import { Outlet } from 'react-router-dom';
3
4  function Dashboard() {
5    |   return <div>Dashboard page <hr /> <Outlet /></div>;
6    | }
7
8  export default Dashboard;
```

It acts as a dynamic placeholder inside a parent component where the matching child route's content will be automatically rendered.

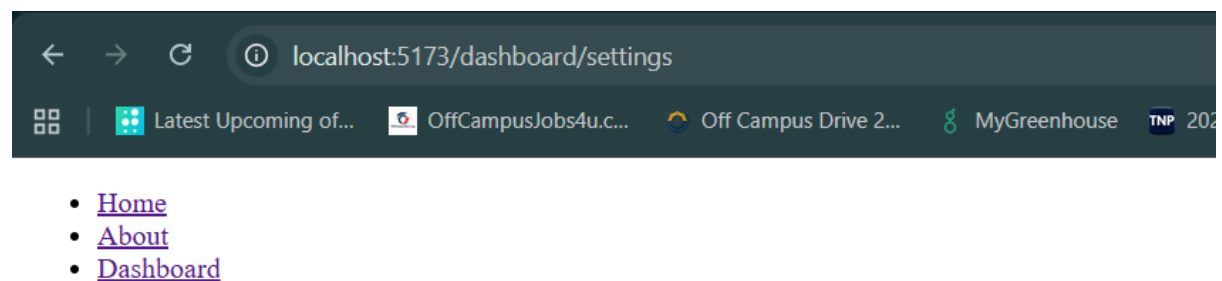
Output:



Dashboard page

Overview Content

Output: for /settings



Dashboard page

Settings Content

Also,

Component	Purpose
Route	Defines one URL path and what to render
Routes	Groups Routes and chooses the matching one
Route inside Routes	Required in React Router v6+

Also,

- **BrowserRouter:** Enables client-side routing in a React application using the browser's History API.
- **Routes:** Acts as a container for all Route components and renders the component that matches the current URL.
- **Route:** Defines a specific URL path and the component that should be displayed when that path is matched.
- **Link:** Used to navigate from one page to another without reloading the entire page.

Also,

Client-Side Routing

- Routing is handled by the browser using JavaScript.
- The page does not reload when navigating between routes.
- Only the required component or content is updated.
- Provides a faster and smoother user experience.
- Commonly used in React applications with libraries like React Router.

Server-Side Routing

- Routing is handled by the web server.
- Each time a user navigates to a new URL, the browser sends a request to the server.
- The server processes the request and returns a new HTML page.
- The entire page reloads on every navigation.

useRef Hook:

What is useRef?

The useRef Hook in React is used to store a value that persists between renders without causing a re-render when it changes.

When to use useRef?

It's commonly used for:

1. Accessing DOM elements directly.
2. Storing mutable values (timers, previous values, etc.).
3. Preserving values between renders without triggering updates.

Syntax:

```
import { useRef } from "react";

const myRef = useRef(initialValue);
```

Example: without useRef.

App.jsx:

```
src > App.jsx > App
1  import React from 'react'
2
3  function App() {
4    return (
5      <div>
6        <h1>useRef Hook</h1>
7        <input type='text' placeholder='Enter user name' />
8        <button>Focus on input field</button>
9      </div>
10   )
11 }
12
13 export default App
14
```

Output:

useRef Hook

Enter user name

Focus on input field

Example: now making the input field get focused when someone click on the button, using the useRef.

App.jsx:

```
src > App.jsx > App > inputHandler
1  import React, { useRef } from 'react'
2
3  function App() {
4    const inputRef = useRef(null);
5    const inputHandler = () => {
6      console.log(inputRef);
7      inputRef.current.focus();
8      inputRef.current.style.color = 'red';
9      inputRef.current.placeholder = 'Enter password'
10   }
11   return (
12     <div>
13       <h1>useRef Hook</h1>
14       <input ref={inputRef} type='text' placeholder='Enter user name' />
15       <button onClick={inputHandler}>Focus on input field</button>
16     </div>
17   )
18 }
19
20 export default App
```

In this code, `useRef(null)` creates a reference called `inputRef` that is attached to the `<input>` element using `ref={inputRef}`. When the button is clicked, the `inputHandler` function runs and uses `inputRef.current` to directly access the actual input box in the DOM. It then focuses the input field (`focus()`), changes the text color to red (`style.color = 'red'`), and updates the placeholder text to "Enter password". In simple terms, `useRef` lets you directly work with a DOM element without re-rendering the component.

Output:

useRef Hook

Enter user name Focus on input field

Output: when you clicked on the button, and started writing. We see that placeholder is changed, focus came on the input field, and the text colour also got changed to red.

useRef Hook

Enter password Focus on input field

useRef Hook

Aditya Focus on input field

Also, we can use the useRef hook to hide and show, although it is not a good practice to do so.

App.jsx:

```
src > App.jsx > default
1  import React, { useRef } from 'react'
2  function App() {
3    const inputRef = useRef(null);
4    const toggleHandler={()=>{
5      if(inputRef.current.style.display!=='none'){
6        inputRef.current.style.display='none';
7      } else {
8        inputRef.current.style.display='inline';
9      }
10   }
11   return (
12     <div>
13       <h1>useRef Hook</h1>
14       <input ref={inputRef} type='text' placeholder='Enter user name' />
15       <button onClick={toggleHandler}>Toggle input field</button>
16     </div>
17   )
18 }
19 export default App
```

In this code, useRef(null) creates a reference to the <input> element and stores it in inputRef. When the Toggle input field button is clicked, the toggleHandler function checks the input's current display style. If the input is visible (display is not 'none'), it hides the input by setting display = 'none'. If the input is already hidden, it makes it visible again by setting display = 'inline'. In simple words, useRef is being used to directly access the input element and show or hide it without using React state.

Output:

useRef Hook

Enter user name Toggle input field

Output: when clicked on the button, input field hided.

useRef Hook

Toggle input field

Using useRef for hiding works, but it skips React's state system, so useState is preferred for UI visibility changes.

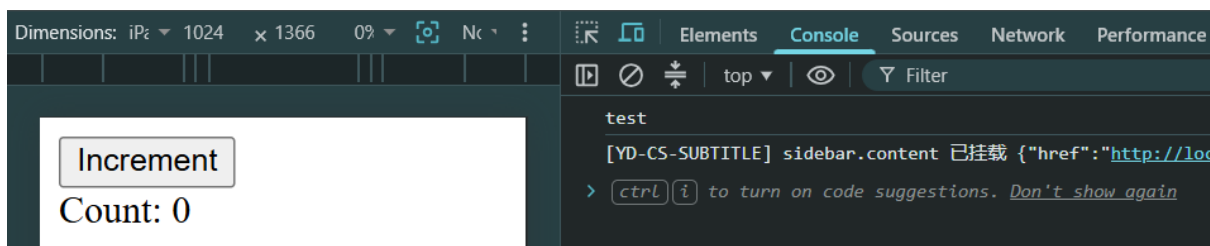
Use case 1: when a variable persists its value across every re-render.

Example: where the variable can't manage to persist its value across re-render. No useRef hook is used here.

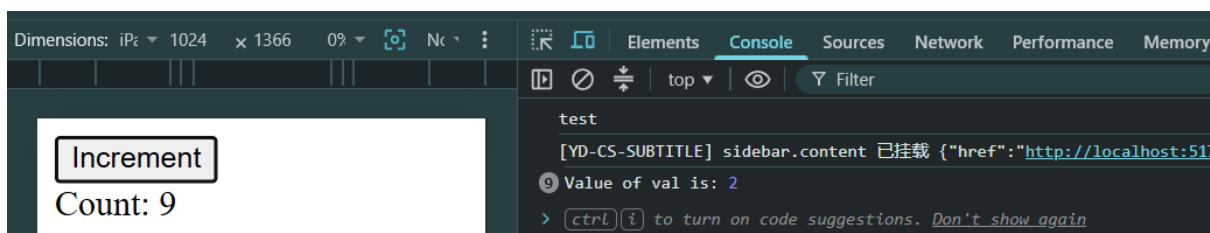
App.jsx:

```
src > App.jsx > ...
1  import React, { useState } from 'react'
2  function App() {
3    const [count, setCount] = useState(0);
4    let val = 1;
5    function handleIncrement(){
6      val = val + 1;
7      console.log("Value of val is:", val);
8      setCount(count+1);
9    }
10   return (
11     <div>
12       <button onClick={handleIncrement}>Increment</button>
13       <br/>
14       Count: {count}
15     </div>
16   )
17 }
18 export default App
```

Output: when loaded.



Output: when clicked on the button for few times, we expected that console shall show that value of val is also increased, but it didn't. It means val can't manage to persist its value on re-render.



In this code, useState is used to manage count, so when the button is clicked, the UI updates correctly. But val is a normal variable, so every time the component re-renders, it resets back to 1, meaning its updated value is not preserved in React. Only count changes the UI because it is stored in state, while val only changes temporarily inside the function and is lost after re-render.

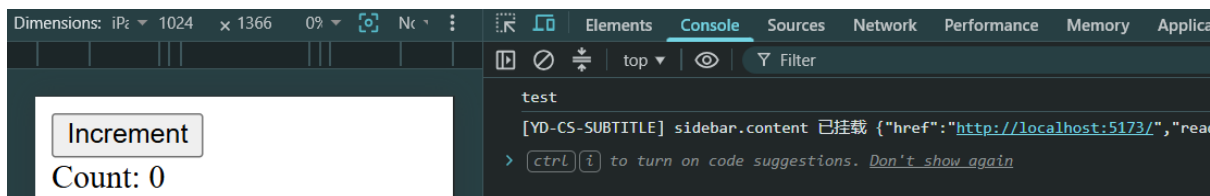
So, what's the solution? Use useRef.

App.jsx:

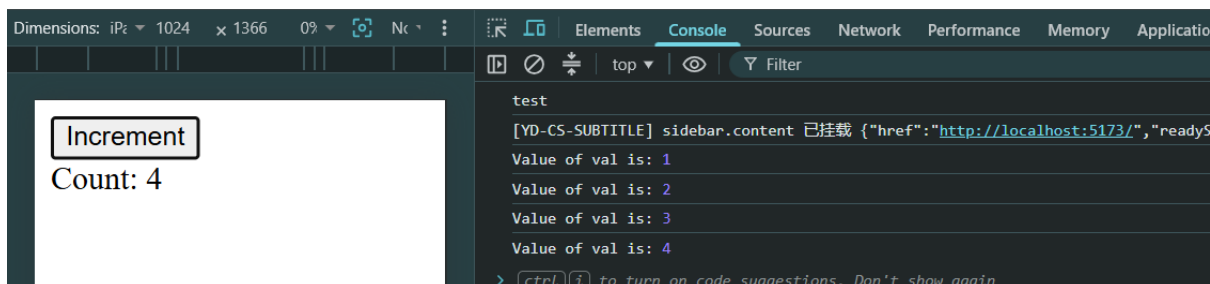
```
src > App.jsx > default
1  import React, { useRef, useState } from 'react'
2  function App() {
3    const [count, setCount] = useState(0);
4    let val = useRef(0);
5    function handleIncrement(){
6      val.current = val.current + 1;
7      console.log("Value of val is:", val.current);
8      setCount(count+1);
9    }
10   return (
11     <div>
12       <button onClick={handleIncrement}>Increment</button>
13       <br/>
14       Count: {count}
15     </div>
16   )
17 }
18 export default App
```

.current is the property inside a useRef object where the actual value is stored.

Output:



Output: when clicked for few times.



In this code, useState is used to manage count, so the UI updates every time the button is clicked. The variable val is created using useRef(0), which stores a value that persists across re-renders without resetting. When the button is clicked, val.current increases and keeps its updated value, and it is logged in the console, while setCount triggers a re-render to update the UI. In simple words, useRef is used here to remember a value between renders without affecting the UI.

Normal Variables vs Variables created using useRef:

Feature	Normal Variable	useRef
Persists after re-render	No	Yes
Causes re-render	No	No
Resets on render	Yes	No
Best use	Temporary logic	Storing stable values (counter, DOM, timers)

Use case 2: When we want to access a DOM element directly.

App.jsx:

```
src > App.jsx > App
1  import React, { useRef, useState } from 'react'
2  function App() {
3    const [count, setCount] = useState(0);
4    let val = useRef(0);
5    let btnRef = useRef();
6    function handleIncrement(){
7      val.current = val.current + 1;
8      console.log("Value of val is:", val.current);
9      setCount(count+1);
10   }
11   function changeColor(){
12     btnRef.current.style.backgroundColor = "red";
13   }
14   return (
15     <div>
16       <button ref={btnRef} onClick={handleIncrement}>Increment</button>
17       <br/>
18       <button onClick={changeColor}>Change colour of 1st button</button>
19       <br/>
20       Count: {count}
21     </div>
22   )
23 }
24 export default App
```

Output:

Increment

Change colour of 1st button

Count: 0

Output: when 2nd button was clicked.

Increment

Change colour of 1st button

Count: 0

In this code, `useState` is used to manage the count value so the UI updates when the button is clicked, while `useRef` is used for two different purposes: `val` stores a value that persists across renders without affecting the UI, and `btnRef` is used to directly access the first button element in the DOM. When the first button is clicked, `handleIncrement` updates both the ref value and the state, and when the second button is clicked, `changeColor` directly changes the background color of the first button to red using `btnRef.current`, showing how `useRef` can be used for direct DOM manipulation.

uncontrolled component:

What is uncontrolled component?

In React, an uncontrolled component is a form element (like an input, textarea, or select) where the DOM itself manages the state, rather than React. Instead of storing the input value in React state and updating it on every change, you let the browser handle it, and you read the value only when needed (usually using a ref).

Key characteristics

- The DOM stores the form data
- React does not control the value in real time
- You typically use ref to access the value
- Less code for simple forms

Controlled vs Uncontrolled component:

Feature	Controlled Component	Uncontrolled Component
State location	React state	DOM
Updates	On every keystroke via <code>onChange</code>	No React updates during typing
Data access	Always in state	Access via ref
Complexity	More code	Simpler
Use case	Dynamic validation, live UI updates	Simple forms, quick input capture

When to use uncontrolled components?

Use them when:

- You don't need instant validation or live updates
- You want simpler code
- You're integrating with non-React libraries
- You only need values on submit

Example: a form controlled by DOM (basically uncontrolled component)

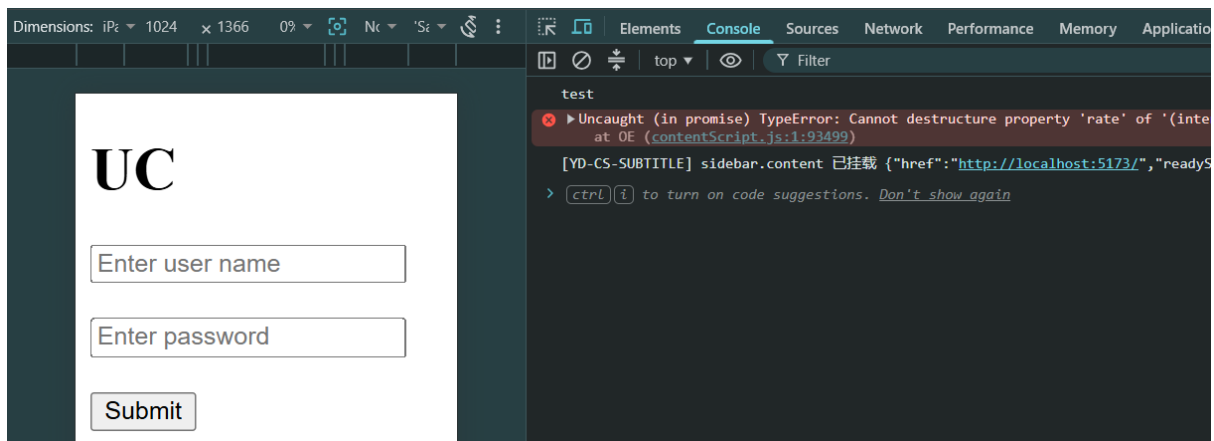
App.jsx:

```
src > App.jsx > App
1 import React from 'react'
2 function App() {
3   const handleForm=(event)=>{
4     event.preventDefault();
5     const user = document.querySelector("#user").value;
6     const password = document.querySelector("#password").value;
7     console.log(user,password);
8   }
9   return (
10    <div>
11      <h1>UC</h1>
12      <form action="" method="post" onSubmit={handleForm}>
13        <input type='text' id='user' placeholder='Enter user name' />
14        <br/>
15        <br/>
16        <input type='password' id='password' placeholder='Enter password' />
17        <br/>
18        <br/>
19        <button>Submit</button>
20      </form>
21    </div>
22  )
23 }
24 export default App
```

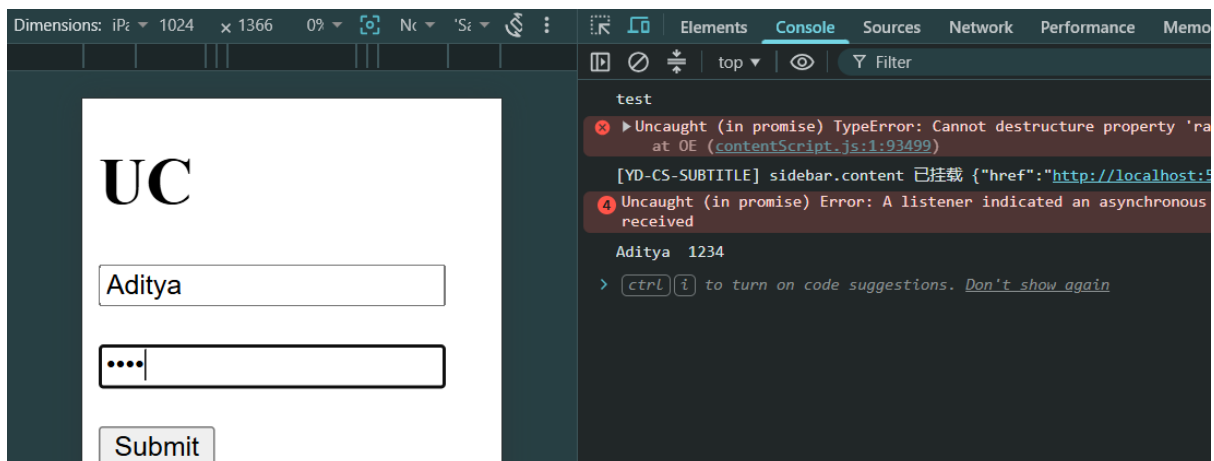
Why is it uncontrolled?

Because the values of the inputs are **not stored in React state**.

Output: ignore those errors, they are because of the extension I have.



Output: fill it and then submit. We can see the entered value in the console.



This component renders a username and password form. When the form is submitted, `handleForm` prevents the page refresh, gets the input values directly from the DOM using `document.querySelector()`, and prints them in the console. Since React does not store the input values in state, it is an uncontrolled component.

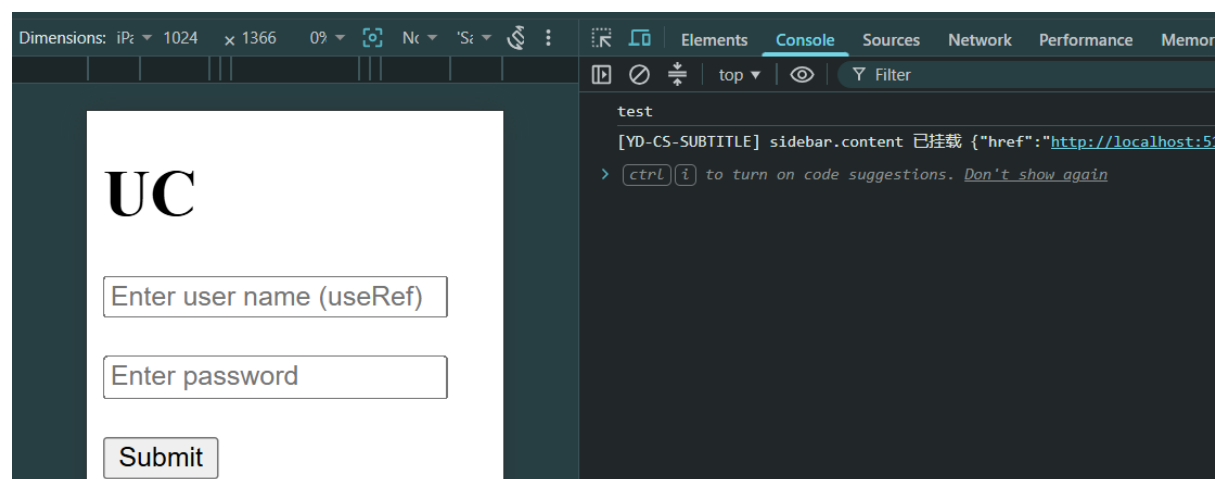
Now, handling the same thing using the **`useRef`**.

App.jsx:

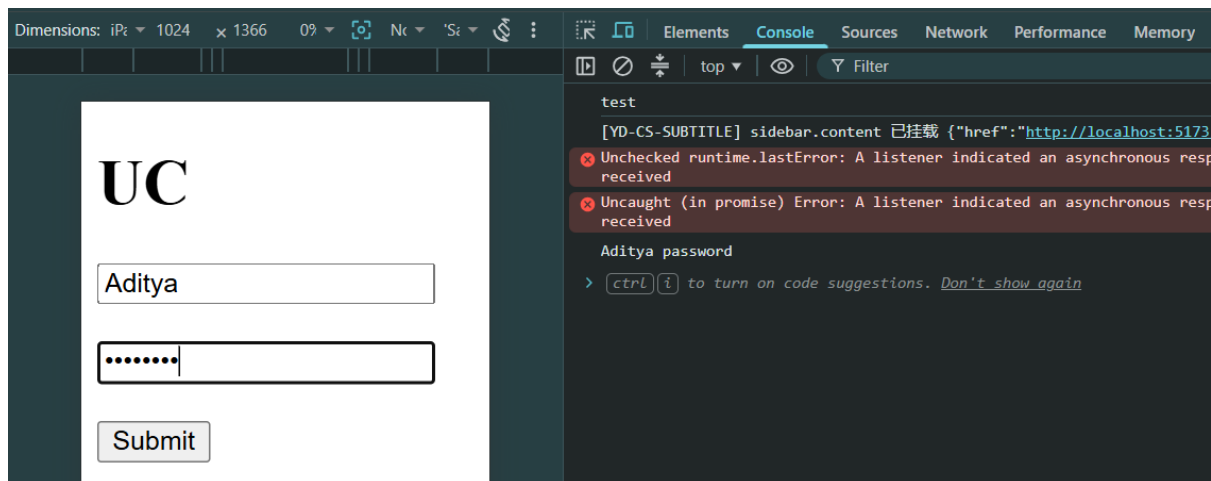
```
src > App.jsx > App
1  import React, { useRef } from 'react';
2  function App() {
3    const userRef = useRef();
4    const passwordRef = useRef();
5    const handleForm = (event) => {
6      event.preventDefault();
7      const user = userRef.current.value;
8      const password = passwordRef.current.value;
9      console.log(user, password);
10   };
11   return (
12     <div>
13       <h1>UC</h1>
14       <form onSubmit={handleForm}>
15         <input type="text" ref={userRef}
16           placeholder="Enter user name"/>
17         <br />
18         <input type="password" ref={passwordRef}
19           placeholder="Enter password"/>
20         <br />
21         <button>Submit</button>
22       </form>
23     </div>
24   );
25 }
26 export default App;
```

`useRef` creates references to the input elements, and `userRef.current.value` and `passwordRef.current.value` give you their current values when the form is submitted. This is still an uncontrolled component because React state is not managing the input values.

Output:



Output:



Now, some more examples of the useRef:

App.jsx:

```
✓ import React from 'react'
  import ColorChanger from './myComponents/ColorChanger'

✓ function App() {
✓   return (
✓     <div>
      <ColorChanger/>
    </div>
  )
}

export default App
```

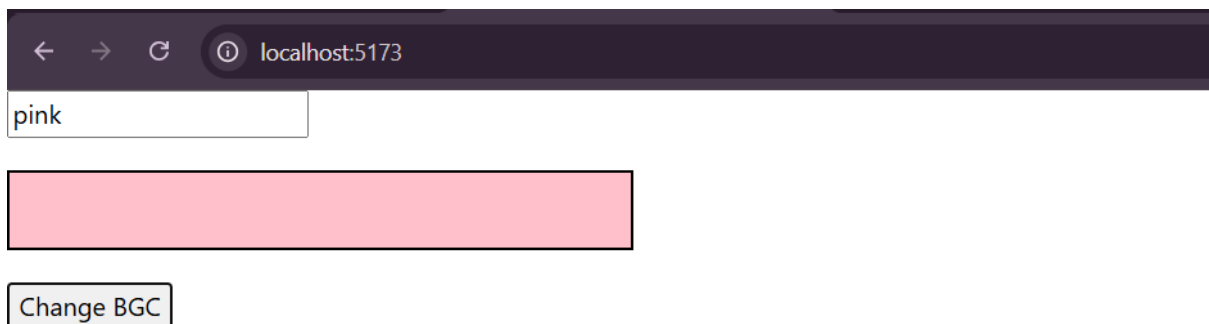
ColorChanger.jsx:

```
src > myComponents > ColorChanger.jsx > default
1  import React, { useRef, useState } from 'react';
2  const ColorChanger = () => {
3    const inputRef = useRef(null);
4    const [bgColor, setBgColor] = useState('');
5    const handleBgChange = () => {
6      setBgColor(inputRef.current.value);
7    };
8    return (
9      <div>
10       <input ref={inputRef} type="text" placeholder=" colour name"/>
11
12       <div style={{ border: '2px solid black',
13         width: '390px',
14         height: '50px',
15         margin: '20px 0',
16         backgroundColor: bgColor
17       }}
18     ></div>
19     <button onClick={handleBgChange}>
20       Change BGC
21     </button>
22   </div>
23 );
24 };
25 export default ColorChanger;
```

Output:



Output: when I wrote the colour name and clicked on the “Change BGC” button.



Pass Function in Component as Props:

Passing a function as a prop in React means sending a function from a parent component to a child component, so the child can execute some logic that belongs to the parent. This is one of the most important concepts in React because it allows child components to communicate with parent components.

Example: normally what we do?

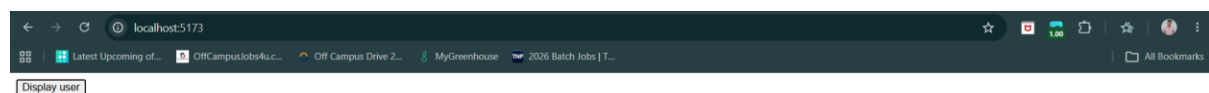
App.jsx:

```
src > App.jsx > App
1  import React from 'react'
2  import User from './myComponent/User'
3
4  function App() {
5    return (
6      <div>
7        <User/>
8      </div>
9    )
10 }
11
12 export default App
13
```

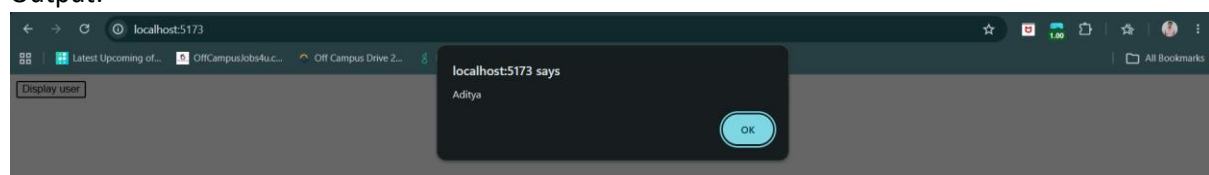
User.jsx:

```
src > myComponent > User.jsx > ...
1  import React from 'react'
2  function User() {
3    const displayName = (name) => {
4      alert("Aditya");
5    }
6    return (
7      <div>
8        <button onClick={displayName}>Display user</button>
9      </div>
10 )
11 }
12 export default User
```

Output:



Output:



Observe, when we do one `<User/>` in the App.jsx, the whole code is rendered once, while if we do multiple `<User/>`, it will get rendered multiple times. This in long run is not a good idea. Instead we will use the function inside the App.jsx, as given below:

```

src > App.jsx > ...
1  import React from 'react';
2  import User from './myComponent/User';
3  function App() {
4    const displayName = () => {
5      alert("Aditya");
6    };
7
8    return (
9      <div>
10       <User displayName={displayName} />
11       <User displayName={displayName} />
12       <User displayName={displayName} />
13     </div>
14   );
15 }
16
17 export default App;

```

User.jsx:

```

src > myComponent > User.jsx > [⌘] default
1  import React from 'react';
2  function User(props) {
3    return (
4      <div>
5        <button onClick={props.displayName}>
6          Display user
7        </button>
8      </div>
9    );
10 }
11 export default User;

```

Output:

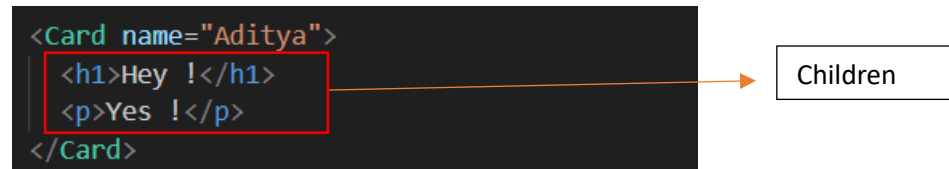
Display user
Display user
Display user

Passing PROPS as Children and Functions:

What is Children?

The children prop is a special prop in React that contains everything you put between a component's opening and closing tags.

Example:



Now, we can see this usage using a basic example.

App.jsx:

```
src > App.jsx > default
1  import React from 'react'
2  import Card from './myComponent/Card'
3
4  function App() {
5    return (
6      <div>
7        <Card name="Aditya">
8          <h1>Hey !</h1>
9          <p>Yes !</p>
10        </Card>
11      </div>
12    )
13  }
14
15  export default App
```

Card.jsx:

```
src > myComponent > Card.jsx > default
1  import React from 'react';
2
3  function Card(props) {
4    return (
5      <div>
6        {props.children}
7      </div>
8    );
9  }
10
11  export default Card;
```

props.children gives you access to whatever is written inside a component's tags.

Output:

Hey !

Yes !

`props.children` is a special React prop that represents the content placed inside a component, and `{props.children}` renders that content at that location in the component's JSX.

Since, there is another way to create the children, using the “children=”.

App.jsx:

```
src > App.jsx > ...
1  import React from 'react'
2  import Card from './myComponent/Card'
3
4  function App() {
5    return (
6      <div>
7        <Card children="Aditya Kumar">
8          <h1>Hey !</h1>
9          <p>Yes !</p>
10        </Card>
11        <Card children="Utsav Kumar">
12          </Card>
13        </div>
14      )
15    }
16
17  export default App
```

Card.jsx:

```
src > myComponent > Card.jsx > default
1  import React from 'react';
2
3  function Card(props) {
4    return (
5      <div>
6        {props.children}
7      </div>
8    );
9  }
10
11  export default Card;
```

Output:

Hey !

Yes !

Utsav Kumar

Now, we will learn how to pass a function from parent to child.

App.jsx:

```
src > App.jsx > default
1  import React, { useState } from 'react';
2  import Button from './myComponent/Button';
3
4  function App() {
5    const [count, setCount] = useState(0);
6
7    function handleClick() {
8      setCount(prev => prev + 1);
9    }
10
11    return (
12      <div>
13        <h1>{count}</h1>
14
15        <Button
16          incrementCount={handleClick}
17          text="Click me"
18        />
19      </div>
20    );
21  }
22
23  export default App;
```

This code creates a React app that keeps track of a number using `useState`. The count variable starts at 0, and the `handleClick` function increases the count by 1 whenever it is called. The current count is displayed inside an `<h1>` tag. The `Button` component is then used and receives two props: `incrementCount`, which is the function that updates the count, and `text`, which sets the button's label to **"Click me"**. When the button is clicked, the number on the screen increases by one.

Button.jsx:

```
src > myComponent > Button.jsx > default
1  import React from 'react';
2
3  function Button(props) {
4    return (
5      <div>
6        {props.children}
7
8        <button onClick={props.incrementCount}>
9          {props.text}
10        </button>
11      </div>
12    );
13  }
14
15  export default Button;
```

It receives information through props, such as the button text (`props.text`), a function to run when the button is clicked (`props.incrementCount`), and any content placed inside the component (`props.children`). The component displays the child content first and then shows a button. When the user clicks the button, the `incrementCount` function is executed, and the button displays the text passed through `props.text`. Finally, the component is exported so it can be used in other files.

Output:

0

Click me

forwardRef:

What is forwardRef?

In React, forwardRef is a method that allows a parent component to pass a ref to one of its child component's DOM elements. forwardRef is a React API that allows a parent component to pass its ref through a custom child component and access the child's underlying DOM element or component instance.

forwardRef is a React utility that allows a parent component to pass a ref through a custom component to one of its child DOM elements.

Without forwardRef, refs work only on:

- HTML elements (input, div, button, etc.)
- Class components

They don't automatically work on function components.

Example: old way (for versions below React 19). Here, we are coding such that when we click on the button the input field should get the value 100 and it shall also get focused.

App.jsx:

```
src > App.jsx > ...
1  import React, { useRef } from 'react'
2  import UserInput from './myComponent/UserInput';
3
4  function App() {
5    const inputRef = useRef(null);
6    const updateInput={()=>{
7      inputRef.current.value = 100;
8      inputRef.current.focus();
9    }}
10   return (
11     <div>
12       <h1>Forward Ref</h1>
13       <UserInput ref={inputRef} />
14       <button onClick={updateInput}>Update Input Field</button>
15     </div>
16   )
17 }
18 export default App
```

This code creates a ref called inputRef and passes it to the UserInput component. Because UserInput uses forwardRef, that ref points to the actual <input> element inside it. When the button is clicked, updateInput() uses inputRef.current to directly access that input, sets its value to 100, and then focuses it, placing the cursor inside the input box.

UserInput.jsx:

```
src > myComponent > UserInput.jsx > ...
1  import React, {forwardRef} from 'react'
2  function UserInput(props,ref) {
3    return (
4      <div>
5        <input type='text' ref={ref}/>
6      </div>
7    )
8  }
9  export default forwardRef(UserInput);
10
```

This code creates a custom UserInput component that forwards any ref it receives from its parent to the actual <input> element inside it. Normally, if a parent puts ref on a function component like <UserInput ref={myRef} />, React won't know which DOM element the ref should point to. By wrapping the component with forwardRef(UserInput), you're telling React: "If someone passes a ref to UserInput, attach it to this <input> instead." This lets the parent directly access the input DOM element and do things like focus(), change its value, or select its text.

Output:

Output: when button was clicked.

Forward Ref

Example: new way (for versions above and equal to React 19). Same thing we will try to do.

App.jsx: same as above.

```
src > App.jsx > ...
1  import React, { useRef } from 'react'
2  import UserInput from './myComponent/UserInput';
3
4  function App() {
5    const inputRef = useRef(null);
6    const updateInput={()=>{
7      inputRef.current.value = 100;
8      inputRef.current.focus();
9    }}
10   return (
11     <div>
12       <h1>Forward Ref</h1>
13       <UserInput ref={inputRef} />
14       <button onClick={updateInput}>Update Input Field</button>
15     </div>
16   )
17 }
18 export default App
19
```

UserInput.jsx:

```
src > myComponent > UserInput.jsx > ...
1  import React from 'react'
2
3  const UserInput=(props)=> {
4    return (
5      <div>
6        <input type='text' ref={props.ref}/>
7      </div>
8    )
9  }
10
11 export default UserInput
12
```

Output:

Forward Ref

Output: when button was clicked.

Forward Ref